

A dependable hiring pipeline

Architecture and design summary · Associate Gen AI Engineer assessment

Problem

A recruiter needs one clear view of candidates for a role, safe stage progression, a reviewable decision history, and a practical way to find people using plain-language questions.

Solution

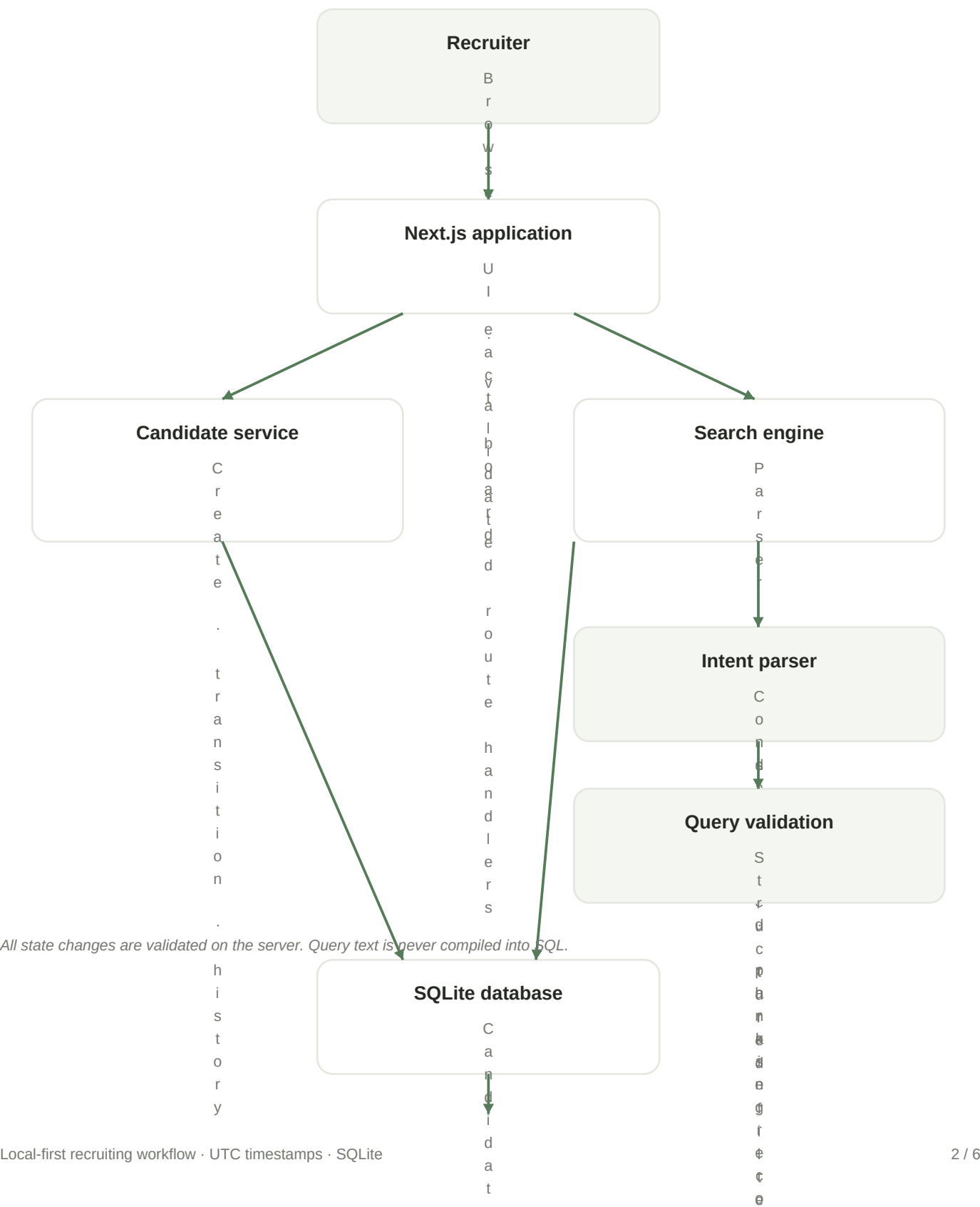
A single Next.js application backed by local SQLite. The board reads the current candidate projection; candidate changes append an immutable event; a constrained search parser maps supported phrases to safe, deterministic filters.

Key requirements

- Five ordered stages plus rejection before hire
- Server side transition validation: one step forward only
- Append only UTC audit events, protected by SQLite triggers
- Fuzzy candidate name lookup and stage, time, history, and exclusion filters
- No paid service, external model, API key, or cloud dependency

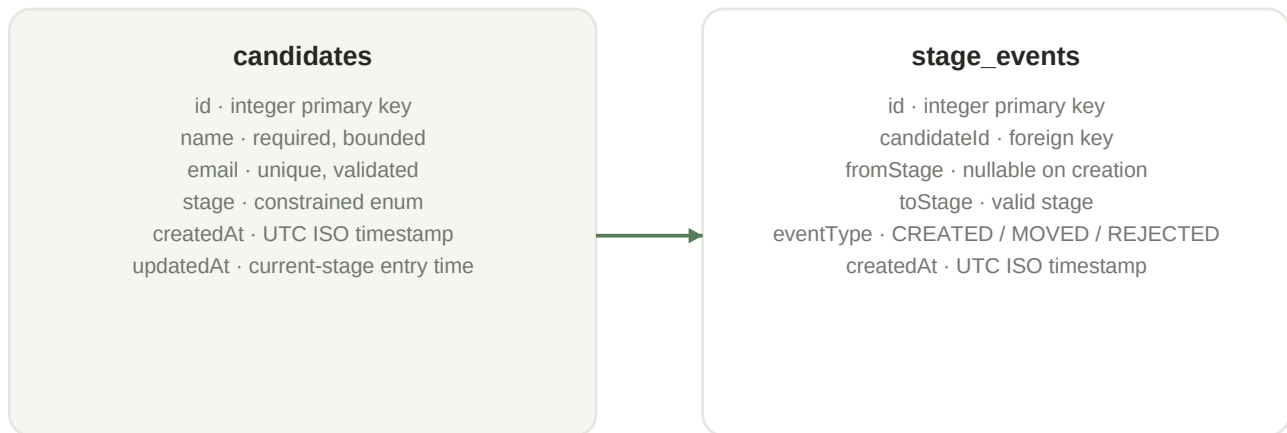
Application architecture

A thin web layer delegates business rules and persistence to local modules.



Data model and audit trail

Current state is convenient to read; events retain the full history.



Atomic writes

Candidate creation inserts the initial APPLIED event in the same transaction. A stage change updates the candidate projection and appends its event in one transaction. A failure cannot leave half of a transition recorded.

Immutability

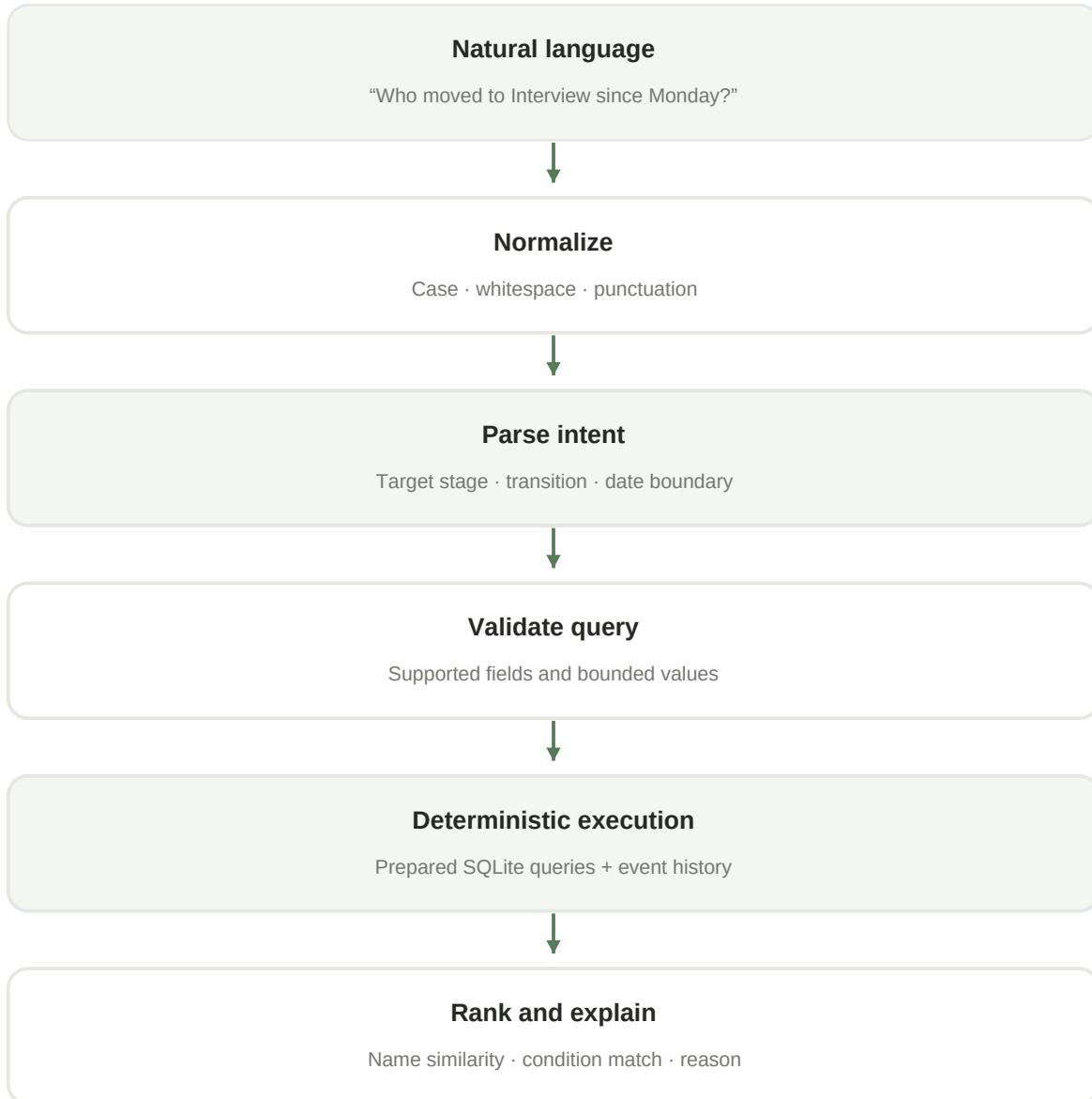
SQLite triggers reject UPDATE and DELETE against **stage_events**, including accidental future code paths. Foreign keys are enabled. History is read in timestamp and event-id order; the current-stage duration is calculated from **updatedAt**.

Indexes

Candidate stage and name; event candidate ID, target stage, and timestamp. This keeps common board and history lookups small and direct.

Natural language search

A bounded grammar turns supported phrases into explainable local queries.



Name ranking: exact full name → exact token → substring/email → local Levenshtein typo match.

Rules, security, and design judgment

Reliability comes from small, explicit rules with testable boundaries.

Stage transitions

- Applied → Screening → Interview → Offer → Hired
- Rejection is allowed from Applied, Screening, Interview, or Offer.
- Hired and Rejected are final outcomes. Skips and reversals are rejected server side.

Security and tests

Zod validates request input. Prepared statements bind all values. No query string becomes SQL. Browser-facing errors stay concise. Unit and query tests cover progression, final outcomes, fuzzy names, stage filters, exclusions, offer history, and unsupported requests.

AI-assisted design decision

AI coding assistance was used during implementation. An AI-assisted design discussion suggested an LLM-first approach that would translate recruiter questions directly into SQL. I chose a constrained parser and validated execution for the critical path because dates, stage rules, duration, exclusions, and audit queries should stay deterministic and testable. No LLM runs in the application.

Trade-offs and next steps

Built for an understandable local demo; scope is intentionally focused.

Future improvements

- Browser-level accessibility and end-to-end tests
- More date phrase coverage and explicit parsed-filter previews
- Schema migrations and safe multi-user concurrency
- Optional intent normalization behind a strict validated schema

Limitations

Single-role, single-user local SQLite demo. No authentication, hosted synchronization, document storage, integrations, or runtime generative AI. Search understands a documented English subset; Monday boundaries use UTC.

Repository

<https://github.com/Pranjal3002/mini-hiring-pipeline>

Run locally: `npm install` · `npm run db:seed` · `npm run dev`